

2024 Sep Oppe 1 - Set 1

Section 1 - problem 1

Sum of squares and absolute difference of squares

PROBLEM STATEMENT

Write a function

sum_squares_abs_diff_squares that takes two integer inputs, a and b, and returns a tuple with:

1. The sum of the squares of a and b.
2. The absolute difference of the squares of a and b.



EXAMPLE & EXPLANATION

Given the inputs a = 3 and b = 4:

Explanation:

- The sum of squares would be $9 + 16 = 25$.
- The absolute difference of squares would be $|9 - 16| = 7$.

So, **sum_squares_abs_diff_squares(3, 4)** should return (25, 7).

The Problem Specification

INPUT

`a (int)`
`b (int)`

Two integer arguments.



THE BLACK BOX

1. `Square` (`a -> a2`)
2. `Square` (`b -> b2`)
3. `Sum` (`a2 + b2`)
4. `Absolute Difference`
`|a2 - b2|`



OUTPUT

`(sum, diff)`

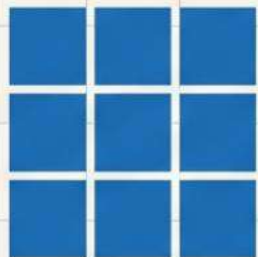
Single Tuple Return Structure.

Constraint: The output must be returned as a single data structure (tuple), not two separate returns.

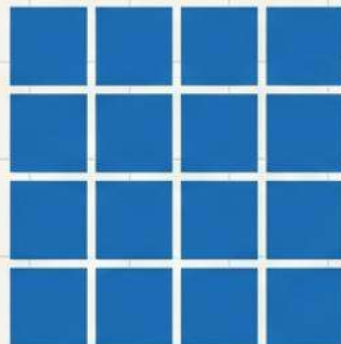
Visualizing the Operations

Example Data: $a = 3$, $b = 4$

The Squares



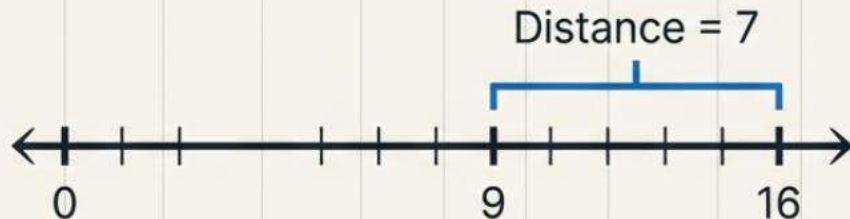
$$3^2 = 9$$



$$4^2 = 16$$

$$9 + 16 = 25 \text{ (The Sum)}$$

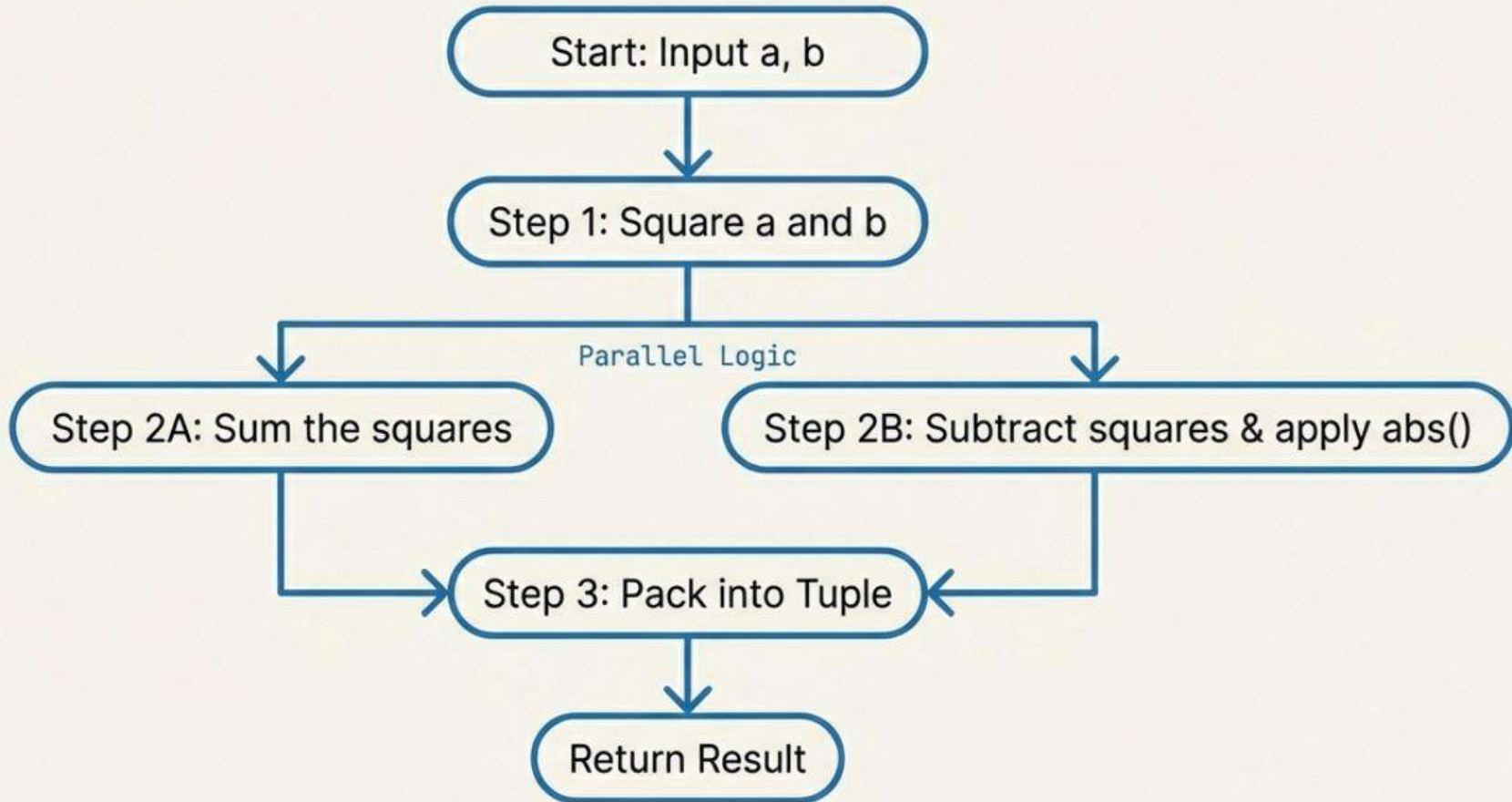
The Difference



$$9 - 16 = -7 \rightarrow \text{abs}(-7) = 7$$

Result Tuple: (25, 7)

The Logic Flowchart



The Explanatory Approach

Step-by-Step implementation for clarity.

```
def sum_squares_abs_diff_squares(a, b):
```

```
# Step 1: Calculate squares
```

```
sq_a = a ** 2
```

```
sq_b = b ** 2
```

Intermediate variables
improve debugging.

```
# Step 2: Calculate sum
```

```
sum_val = sq_a + sq_b
```

```
# Step 3: Calculate absolute difference
```

```
diff_val = abs(sq_a - sq_b)
```

Ensures positive
integer result.

```
return (sum_val, diff_val)
```

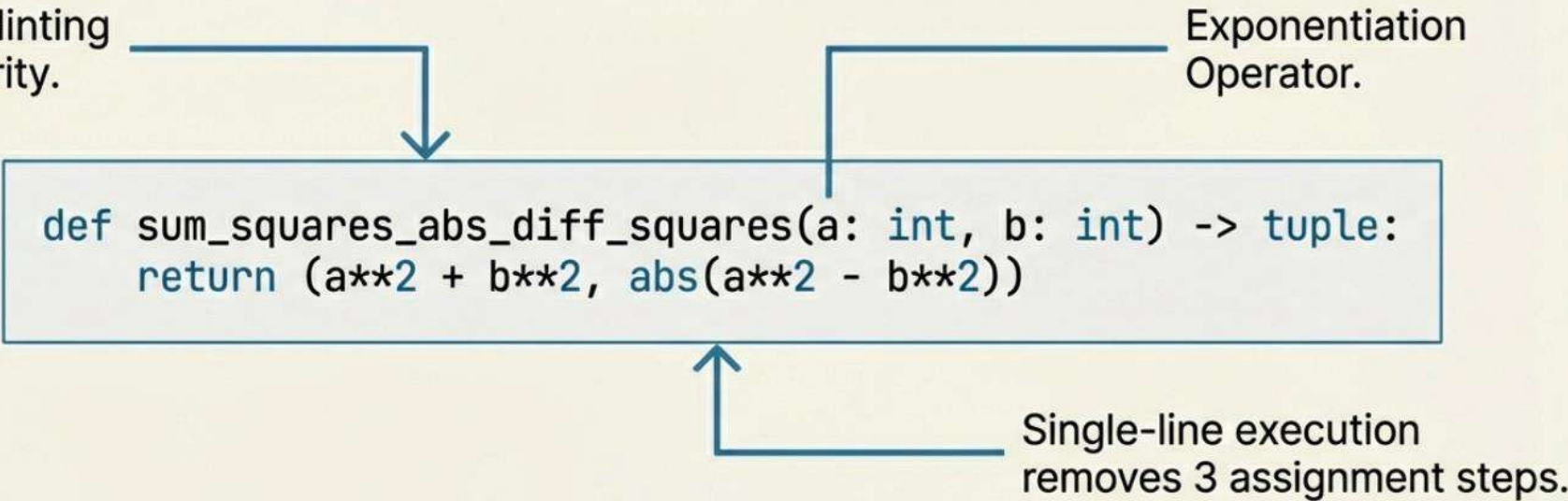
Explicit tuple construction.

The Pythonic Optimization

Leveraging built-ins for concise logic.

Type Hinting
for clarity.

Exponentiation
Operator.



```
def sum_squares_abs_diff_squares(a: int, b: int) -> tuple:  
    return (a**2 + b**2, abs(a**2 - b**2))
```

Single-line execution
removes 3 assignment steps.

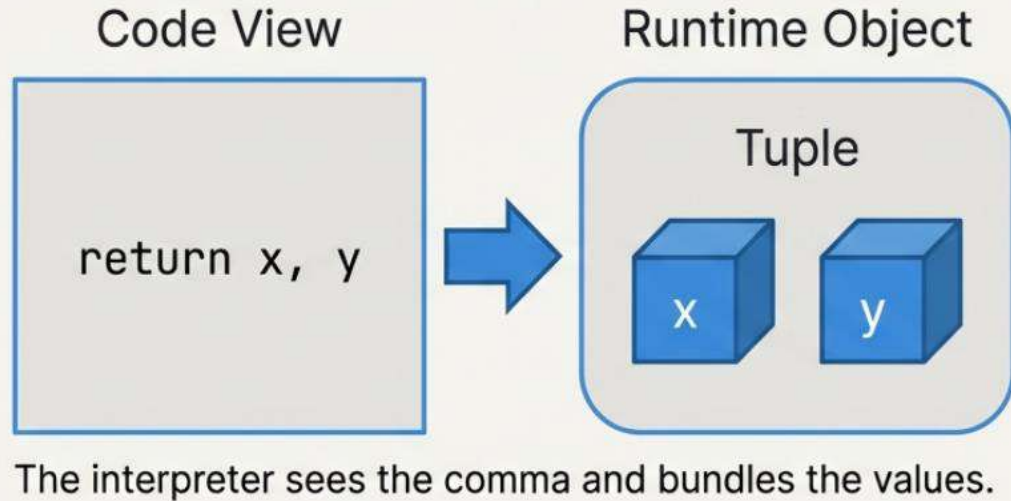
Result: Identical output, cleaner syntax.

Concept Spotlight: Tuple Packing

Leveraging the comma as a constructor.

In Python, the comma is the tuple constructor, not just the parentheses.

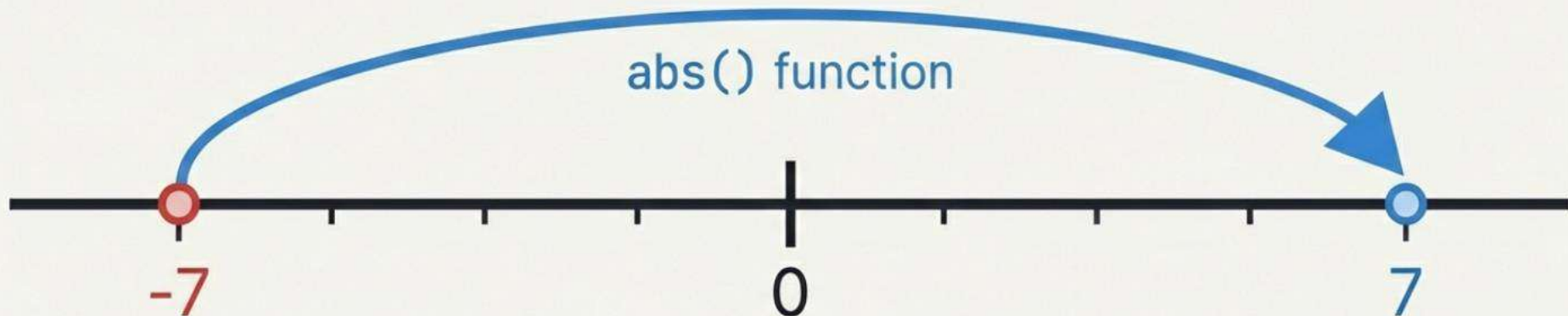
Returning multiple values separated by commas automatically packs them into a single tuple object.



Application: `(sum, diff)` instantly creates the required data structure.

Concept Spotlight: The Power of abs()

Why magnitude matters more than direction.



Scenario A:

$16 - 9 = 7$ (Already Positive)

Scenario B:

$9 - 16 = -7$ (Negative)

Correction:

$\text{abs}(-7)$ transforms to 7.

Robustness Rule: Always use `abs()` for 'difference' problems to prevent logic errors on variable input.

Solution Comparison

Step-by-Step Approach



- High Readability.
- Clear intermediate steps.
- Best for: Beginners & Debugging.

```
sq_a = a**2  
sq_b = b**2  
return (sq_a + sq_b)**0.5
```

Concise One-Liner



- Mathematical purity.
- Fewer memory assignments.
- Best for: Production & Competitive Coding.

```
return (a**2 + b**2)**0.5
```

Key Takeaways

01

Return Types

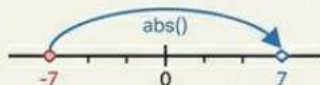
Returning 'x, y' creates a tuple automatically. Parentheses are optional but recommended for readability.

(x, y)

02

Mathematical Safety

Wrap difference calculations in `abs()` whenever the direction of subtraction is irrelevant.



03

Efficiency

Direct calculation in the return statement reduces noise for simple arithmetic operations.



Final Syntax Cheat Sheet

```
return (a**2 + b**2, abs(a**2 - b**2))
```